VAULTPILOT AI: FINANCIAL EVIDENCE INVESTIGATION FOR NGOS

UGREEN Hackathon 2026 on AI NAS | Official Round 1 Project Proposal

Track: Workflow / Innovation | Skill ID: vaultpilot | Standard: UGREEN UGOS / Uliya v0.7.3

1. What does your Skill do? (Must be answered in just one sentence)

VaultPilot AI is a NAS-native financial evidence investigation Skill that discovers, extracts, and cross-reconciles purchase orders, invoices, and payment receipts stored on UGREEN AI NAS to detect documentation variances and evidential gaps without altering source records.

2. Who is your target user?

Non-governmental organizations (NGOs), development agencies, and compliance auditors managing grant-funded project expenditures across decentralized local storage.

3. What is their problem?

Grant recipients face financial clawbacks during compliance audits due to missing payment receipts, unauthorized purchase order variances, and unindexed amendments scattered across unorganized folder trees. Manual folder verification is labor-intensive, error-prone, and risks cloud data leakage of sensitive donor budgets.

4. What is your solution?

An on-device evidence reconstruction workflow that performs multi-document three-way matching, enforces a conservative non-accusatory audit taxonomy, and provides human-in-the-loop verification with immutable, cryptographically sealed investigation reports.

5. What is the main user flow?

1. User specifies authorized NAS directory scope. 2. System indexes financial files. 3. System extracts structured transaction entities. 4. Engine evaluates three-way reconciliation (PO vs Invoice vs Receipt). 5. Discrepancies and gaps are flagged. 6. Auditor reviews, verifies, or corrects fields. 7. System re-reconciles transactions dynamically. 8. Official report is sealed with SHA-256 integrity snapshot.

6. How is your idea different from existing tools or common AI solutions?

Unlike single-document OCR or generic chatbots, VaultPilot AI links fragmented records across four tiers (*Confirmed Discrepancy*, *Potential Anomaly*, *Missing Evidence*, *Unverified*). It never makes speculative fraud allegations. It segregates multi-currency totals and guarantees that report snapshots remain cryptographically immutable after review adjustments.

7. Which UGREEN AI NAS features does your Skill use, and at which step? Describe your technical approach.

- Discovery: Delegates to ‘file-agent‘ using ‘ug-filesystem.find‘ within user-authorized paths. - Extraction & Comparison: Delegates to ‘document-agent‘ using ‘ug-document.extract‘ and provisional ‘ug-document.reconcile‘. - Technical Approach: Declarative Markdown orchestration under Uliya v0.7.3 utilizing the onboard UGOS LLM. Operates strictly read-only with passive untrusted text handling and zero cloud dependencies.

8. What have you built so far, what is not built yet, and what still needs testing on a NAS?

- Built: Declarative Skill package, three-way multi-source reconciliation engine, human review interface, immutable reporting with SHA-256 snapshots, and 155 automated backend tests. - Not Built: Multi-tenant role-based access control. - Needs NAS Testing: Execution of provisional CLI capabilities and file discovery performance on physical UGOS hardware (‘iDX6011 Pro‘).

9. How will you test your Skill if you become a Finalist? Name two core test cases and one edge case.

- Core 1: Complete three-way match (PO = Invoice = Receipt) yielding verified expenditure. - Core 2: Documented overbilling variance resolved through human auditor review correction of invoice billing based on a credit memo. - Edge Case: Degraded, unreadable receipt scan classified as *Unverified* with zero numerical hallucination and zero spend credited.

10. What are the known limits, risks, or outside dependencies of your Skill?

- Limits & Risks: All CLI signatures are provisional design mappings pending Round 2 hardware validation. Physical NAS and Uliya runtime execution are unverified. No certified technical quarantine or prompt-injection immunity is claimed. - Dependencies & Licenses: Skill package contains zero third-party dependencies. Companion application uses FastAPI (MIT), React (MIT), Vite (MIT), SQLAlchemy (MIT), and ‘pdftotext‘ from ‘poppler-utils‘ (GPL-2.0). All processing is local.